



Fault Tolerance in Distributed Systems

Atomic Commitment

slide credits:
Timofei Istomin

Atomic Commitment (AC)

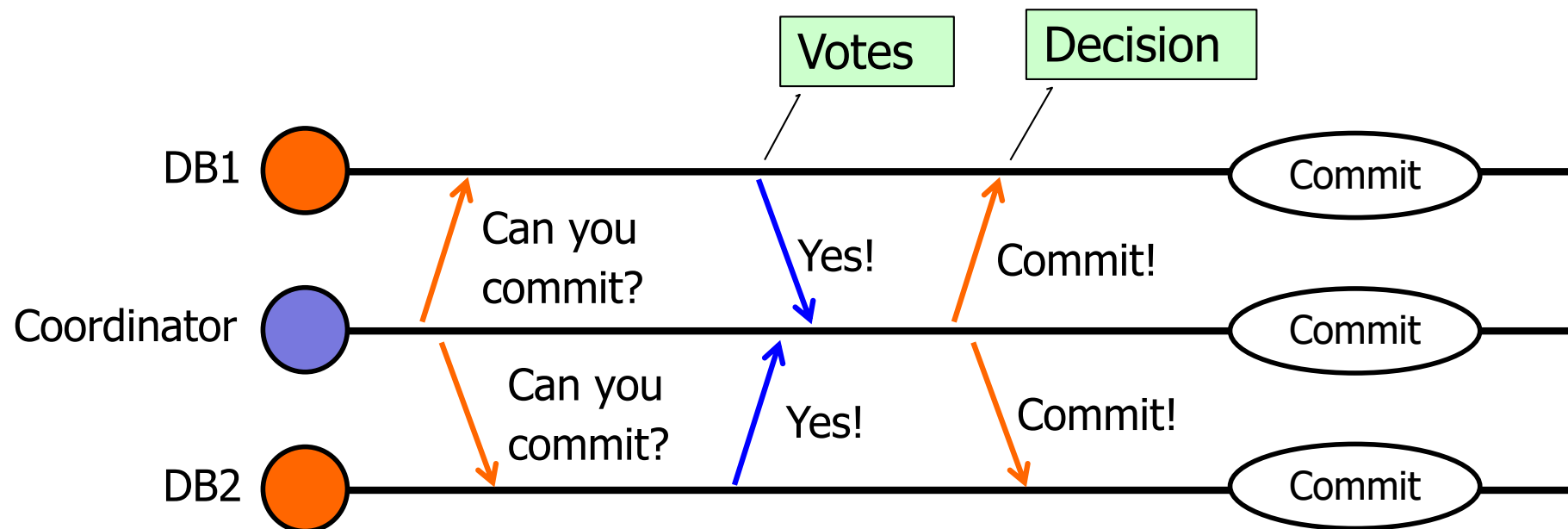


- ◆ A special case of agreement...
- ◆ ... particularly important for distributed databases
- ◆ What's a distributed database?
 - ◆ Two airlines, two separate ticket databases
 - ◆ We want to buy either both parts of the route or neither — the databases should either both commit or both abort
- ◆ **Distributed transaction**
- ◆ Databases have to agree on whether to commit/abort

AC without faults is simple



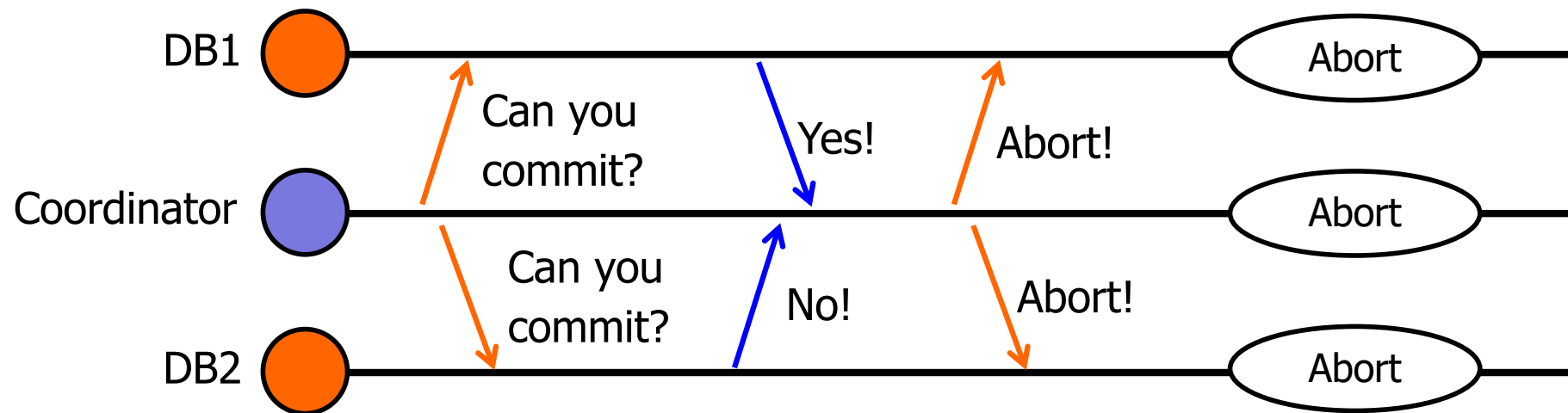
- ◆ A dedicated coordinator process asks the DB servers if they are able to commit the transaction
- ◆ If all answer Yes, it signals them to commit



AC without faults is simple



- ◆ ... if someone cannot commit the transaction, all should abort



AC with faults is tricky



- ◆ What should the system do if a DB process crashes in the middle?
- ◆ What about a coordinator crash?
- ◆ What should the coordinator do if a vote does not arrive in time?
- ◆ What should a participant do if the decision does not arrive in time?
- ◆ We study distributed algorithms that provide answers to these problems

Dealing with crashes



- ◆ Processes may store (log) their decisions and the current step of the protocol to the stable storage
- ◆ If they crash, they can later **recover** by retrieving the state from the log file
- ◆ **We assume that storage never fails!**

This converts crash failures into
response omission + performance failures!

Dealing with packet loss



- ◆ To deal with lost packets we could, e.g.:
 - ◆ Coordinator: re-request periodically the vote from a participant
 - ◆ Participant: re-request periodically the decision from the coordinator

This converts response omissions
into performance failures!

Dealing with slow responses



- ◆ We have converted all failures into performance failures!
- ◆ But how to deal with them?
- ◆ Processes could just wait until the response finally arrives
 - ◆ If it's fine for our customers, we are done!
- ◆ But they may have to wait for a really long time
 - ◆ e.g., when a server must be physically replaced

Blocking



- ◆ We say that a process is **blocked** if it has to wait for a reply from another process before it can complete the algorithm
- ◆ In other words, it cannot simply timeout in case of no-reply and still guarantee *safety*
 - ◆ I.e., in this case, make sure that its decision is coherent with everyone else's
- ◆ Ideally, we want to design a correct **and** non-blocking protocol under all failure modes

Safety

Liveness

Impossibility results



- ◆ But... it is proven that
 - ◆ There are **no** non-blocking AC algorithms in the presence of **communication failures**
 - Due to the “Coordinated attack” problem
 - ◆ There are **no** non-blocking AC algorithms in the presence of **total crash failures**
 - i.e., at least one non-faulty process throughout protocol execution is required
- ◆ AC is **not** solvable with Byzantine failures:
 - ◆ a process might pretend to be correct but, at the end, decide on the opposite of the global decision

Atomic Commitment



◆ Properties:

- ◆ **Uniform agreement:** no two processes (correct or not) decide differently (commit vs. abort)

A process is correct if it never crashed during the whole execution

- ◆ **Validity:** global “commit” decision is possible only if *all* processes voted “yes”

-> If some voted “no”, all *must* ABORT

- ◆ **Non-triviality:** global decision *must* be “commit” if *all* voted “yes” and there are no faults

-> if there are faults, the global decision *may* be ABORT

- ◆ **Termination:** If there are no faults (or there are, but all get repaired), all processes eventually decide

- ◆ **Decision is final:** once decided, cannot change even with failures

- ◆ **Goal:** design a *correct* atomic commitment protocol that is unlikely to block

Two-phase Commit (2PC)



- ◆ It is the most popular algorithm
- ◆ Correct even in asynchronous systems with temporary communication failures
 - ◆ slow reactions become “performance failures”: a fault is caused by a **timeout expiration**
 - a node gives up waiting for a message
 - ◆ this relaxes the **non-triviality** and **termination** requirements
 - more situations where the protocol is allowed to ABORT or nodes can avoid deciding...

How come? In previous classes we said that agreement is impossible in asynchronous systems with crash failures...

Here, crashes are **masked** using reliable storage!

Two-phase Commit

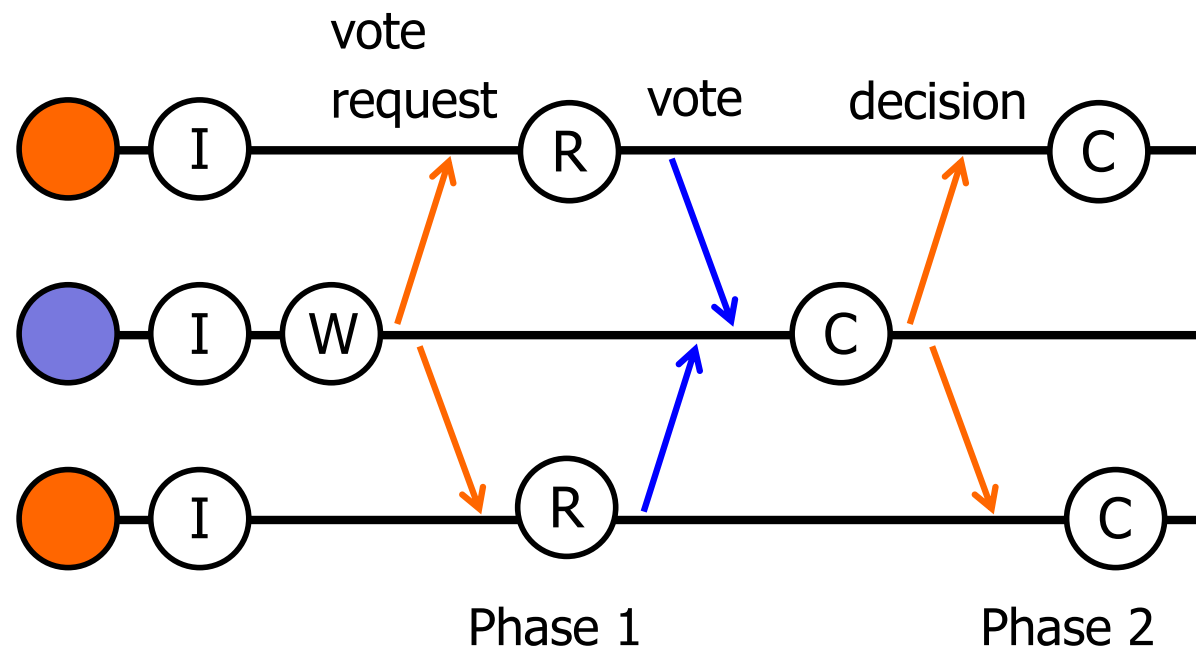


Phase 1: coordinator requests votes, collects responses, and decides COMMIT or ABORT

Phase 2:
coordinator sends out the decision

States:

INIT, WAIT,
READY, COMMIT,
ABORT



Termination



- ◆ Question: when does a process decide (and therefore terminate)?
- ◆ Anyone voting NO decides ABORT immediately, at the beginning of the transaction
 - ◆ Nobody can commit if some processes vote NO...
 - ◆ Processes may decide to vote NO for applicative reasons, e.g., a leg of a multi-leg flight is unavailable
- ◆ Coordinator decides after collecting votes
 - ◆ If it sees a NO, decides ABORT
 - ◆ Otherwise, it decides COMMIT
- ◆ All the other nodes decide when they receive the final decision from the coordinator